

RISC-V (RV32I) Processor Core

Oancea Alexandru

Microarchitecture Specification & Design Documentation

github.com/alex1oancea/riscv-pipelined-core

Table of Contents

RISC-V (RV32I) Processor Core	1
Chapter 1: Introduction	2
Chapter 2: Datapath Architecture	2
2.1. Program Counter (PC)	3
2.2. Register File	3
2.3. Arithmetic Logic Unit (ALU)	3
2.4. Multiplexers and Sign Extension	4
2.5. Critical Path Analysis	4
Chapter 3: Control Unit	4
3.1. Main Decoder	5
3.2. ALU Decoder	5
3.3. Hardware Design Considerations (Latch Prevention)	5
Chapter 4: Memory System and Integration	6
4.1. Instruction and Data Memories	6
4.2. Top-Level Integration	6
Chapter 5: Verification and Hardware Debugging	6
5.1. Self-Checking Testbench	6
.....	7
5.1.2. Waveform Analysis	7
Chapter 6: Pipelined Architecture	8
6.1. The 5-Stage Pipeline	8
6.2. Pipeline Registers	8
Chapter 7: Hazard Resolution (Hazard Unit)	9
7.1. Data Hazards and Forwarding	9

7.2. Load-Use Data Hazards and Stalling.....	9
7.3. Control Hazards and Branch Flushing.....	9
7.4. Hardware Design Considerations: NOP Injection	9
7.5. Pipeline Execution & Hazard Resolution Waveform	10
Chapter 8: Conclusion	10

Chapter 1: Introduction

This document describes the architecture and implementation of a 32-bit processor based on the RISC-V (RV32I) instruction set. The project is developed in the SystemVerilog hardware description language and serves an educational purpose, aiming for a deep understanding of how software instructions are translated into physical hardware signals.

The design and architecture of this processor are heavily based on the concepts and schematics presented in the reference book "**Digital Design and Computer Architecture, RISC-V Edition**" by Sarah L. Harris and David Harris.

The project is implemented iteratively: the initial version represents a Single-Cycle architecture (where each instruction is executed in a single clock cycle). Subsequently, the design will be scaled to a 5-stage Pipelined architecture (Fetch, Decode, Execute, Memory, Writeback), including hazard resolution mechanisms such as Forwarding and Stalling.

Chapter 2: Datapath Architecture

The Datapath represents the physical route that data and addresses take through the processor during the execution of an instruction. It consists of various interconnected functional units that operate based on the signals provided by the Control Unit.

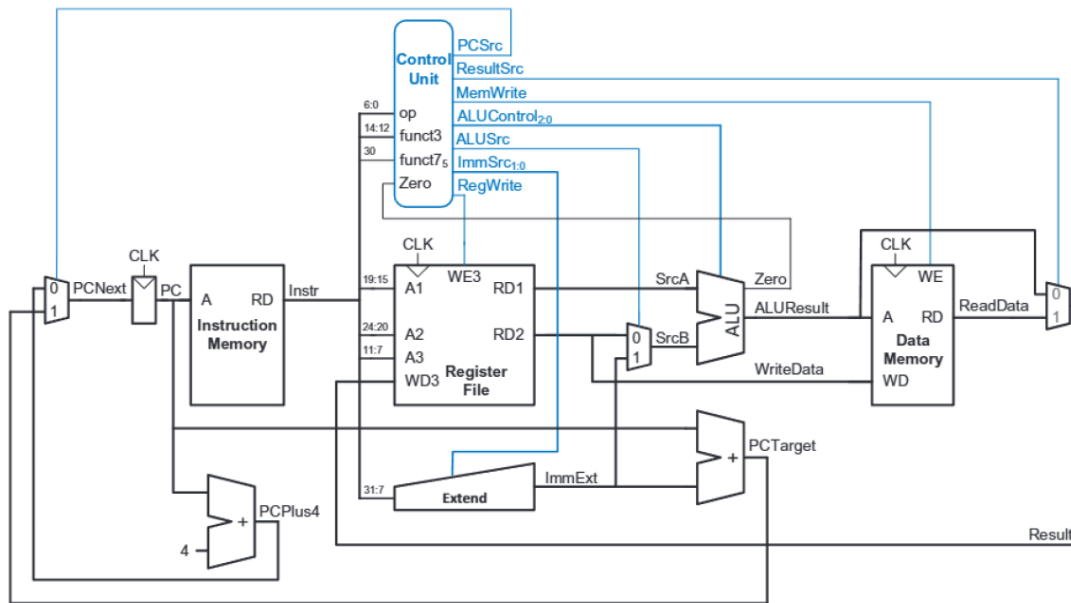


Figure 2.1: Complete single-cycle RISC-V microarchitecture. This schematic illustrates the datapath and control unit routing. (Reference: Harris & Harris, *Digital Design and Computer Architecture RISC-V Edition*).

2.1. Program Counter (PC)

The PC is a 32-bit sequential register that holds the memory address of the current instruction being executed. On every rising edge of the clock (clk), the PC is updated to point to the next instruction. By default, it increments by 4 (since each instruction is 32 bits, or 4 bytes). In the case of branches or jumps, the PC is updated with a calculated target address.

2.2. Register File

The processor features a 32x32-bit Register File. It contains 32 general-purpose registers (x0 through x31), where x0 is hardwired to the constant value 0. The Register File allows two simultaneous asynchronous reads (outputs RD1 and RD2) and one synchronous write (input WD3, controlled by the RegWrite signal).

2.3. Arithmetic Logic Unit (ALU)

The ALU performs mathematical and bitwise operations. It receives two 32-bit operands (SrcA and SrcB) and a 3-bit control signal (ALUControl) from the ALU Decoder. It outputs a 32-bit ALUResult and a Zero flag. The Zero flag evaluates to 1 when the result is 0, which is critical for determining branch outcomes (e.g., beq).

2.4. Multiplexers and Sign Extension

To allow instructions to share datapath resources, multiplexers (MUX) act as physical switches controlled by the Control Unit:

- **ALUSrc MUX:** Selects the second operand for the ALU between a register value (RD2) and an immediate value.
- **ResultSrc MUX (MUX3):** Selects the final data to be written back to the Register File from either the ALU result, Memory Read Data, or PC+4.
- **Extend Module:** Takes immediate values of varying lengths from the instruction bits and sign-extends them to a full 32-bit representation.

2.5. Critical Path Analysis

In the single-cycle architecture, the critical path is dictated by the Load Word (lw) instruction, which requires reading the Register File, accessing the Data Memory, and writing back in one cycle. Post-synthesis timing reports indicate a critical path delay of approximately 7.7 ns, yielding a maximum theoretical frequency of ~130 MHz.

In the pipelined architecture, the physical path is heavily partitioned, which conventionally increases frequency. However, post-synthesis analysis reveals a critical path delay of approximately 8.7 ns (maximum frequency of ~115 MHz). This highlights a classic bottleneck shift in hardware design: the overhead introduced by the complex routing of the Hazard Unit (forwarding multiplexers and stall logic) outweighs the timing benefits of splitting the simplistic ALU and Data Memory. While the single-cycle core completes the benchmark test program in ~185ns, the pipelined processor finishes in ~264ns due to pipeline fill time and dynamically inserted flush cycles.

Consequently, while the single-cycle processor achieves a higher f_{max} , the pipelined architecture remains structurally superior for scaling toward complex systems featuring multi-cycle components or memory hierarchies.

Chapter 3: Control Unit

The Control Unit is the "brain" of the processor, responsible for decoding the instructions fetched from memory and generating the control signals that route data through multiplexers and dictate ALU operations. It is divided into two main sub-modules: the Main Decoder and the ALU Decoder.

3.1. Main Decoder

The mainDecoder module evaluates the 7-bit opcode of the instruction to determine its general category (memory operations, arithmetic operations, branches, etc.). Based on this opcode, the module generates the primary control signals for the Datapath.

Main Decoder Truth Table:

Instruction	Opcode	RegWrite	ImmSrc	ALUsrc	MemWrite	ResultSrc	Branch	ALUOp	Jump
lw	0000011	1	00	1	0	01	0	00	0
sw	0100011	0	01	1	1	Xx	0	00	0
R-type	0110011	1	xx	0	0	00	0	10	0
beq	1100011	0	10	0	0	xx	1	01	0
I-type	0010011	1	00	1	0	00	0	10	0
jal	1101111	1	11	x	0	10	0	xx	1

(Note: "X" values represent "Don't Care" bits, indicating that the state of that specific signal does not affect the execution of the current instruction. This allows for hardware logic optimization during synthesis).

3.2. ALU Decoder

While the Main Decoder identifies the general instruction family, the aluDecoder module is responsible for specifying the exact mathematical or bitwise operation the ALU must execute. It combines the ALUOp signal received from the Main Decoder with the function bits from the instruction (funct3 and, in certain cases, bit 5 of funct7).

ALU Decoder Truth Table:

ALUOp	Funct3	{op5, funct7[5]}	ALUcontrol	Instruction
00	x	x	000(add)	lw,sw
01	x	X	001(sub)	beq
10	000	x	000	add
	000	x	001	sub
	010	x	101	slt
	110	x	011	or
	111	x	010	and

3.3. Hardware Design Considerations (Latch Prevention)

In the SystemVerilog hardware description, both decoders were implemented using combinational logic (always_comb). To ensure the generation of a valid and stable logic

circuit, and to prevent the synthesizer from unintentionally inferring memory elements (*Latches*), the decision blocks (case statements) strictly include a default: clause. This guarantees that for any unexpected or undefined combination of input signals, the outputs fall back to a predictable state, thereby maintaining the integrity of the datapath.

Chapter 4: Memory System and Integration

4.1. Instruction and Data Memories

The architecture follows a Harvard-like structure, separating the instruction and data memory paths to allow simultaneous fetching and data access in a single clock cycle.

- **Instruction Memory (`instrmem`):** Acts as a Read-Only Memory (ROM). It asynchronously outputs the 32-bit instruction located at the address provided by the Program Counter.
- **Data Memory (`datamem`):** Acts as a Random Access Memory (RAM). It handles Load (`lw`) and Store (`sw`) instructions. Reading is asynchronous, while writing is synchronous and strictly gated by the `MemWrite` control signal.

4.2. Top-Level Integration

The top module serves as the highest-level hardware wrapper. It instantiates the RISC-V processor core alongside the `instrmem` and `datamem`.

Chapter 5: Verification and Hardware Debugging

5.1. Self-Checking Testbench

To validate the processor's functionality, a self-checking testbench (`tb.sv`) was implemented. Instead of manually inspecting waveform signals for every clock cycle, the testbench automatically loads a machine-code program (`riscvtest.txt`) and monitors specific memory transactions. Upon completion, it asserts the final expected state.

```

# riscvtest.s
# Sarah.Harris@unlv.edu
# David_Harris@hmc.edu
# 27 Oct 2020
#
# Test the RISC-V processor:
# add, sub, and, or, slt, addi, lw, sw, beq, jal
# If successful, it should write the value 25 to address 100
#
# RISC-V Assembly      Description      Address  Machine Code
main:  addi x2, x0, 5      # x2 = 5          0      00500113
      addi x3, x0, 12     # x3 = 12          4      00C00193
      addi x7, x3, -9     # x7 = (12 - 9) = 3  8      FF718393
      or x4, x7, x2       # x4 = (3 OR 5) = 7  C      0023E233
      and x5, x3, x4      # x5 = (12 AND 7) = 4 10     0041F2B3
      add x5, x5, x4      # x5 = 4 + 7 = 11    14     004282B3
      beq x5, x7, end     # shouldn't be taken 18     02728863
      slt x4, x3, x4      # x4 = (12 < 7) = 0  1C     0041A233
      beq x4, x0, around  # should be taken    20     00020463
      addi x5, x0, 0      # shouldn't execute  24     00000293
around: slt x4, x7, x2     # x4 = (3 < 5) = 1   28     0023A233
      add x7, x4, x5      # x7 = (1 + 11) = 12 2C     005203B3
      sub x7, x7, x2      # x7 = (12 - 5) = 7  30     402383B3
      sw x7, 84(x3)       # [96] = 7          34     0471AA23
      lw x2, 96(x0)       # x2 = [96] = 7      38     06002103
      add x9, x2, x5      # x9 = (7 + 11) = 18 3C     005104B3
      jal x3, end         # jump to end, x3 = 0x44 40     008001EF
      addi x2, x0, 1      # shouldn't execute  44     00100113
end:    add x2, x2, x9     # x2 = (7 + 18) = 25 48     00910133
      sw x2, 0x20(x3)     # [100] = 25         4C     0221A023
done:   beq x2, x2, done  # infinite loop      50     00210063

```

Figure 5.1: Instructions used for verification.

(Reference: Harris & Harris, *Digital Design and Computer Architecture RISC-V Edition*).

5.1.2. Waveform Analysis

The following waveform validates the temporal behavior of the processor during simulation. Key signals such as the Program Counter (PC), fetched instruction (Instr), and ALU results are observable.

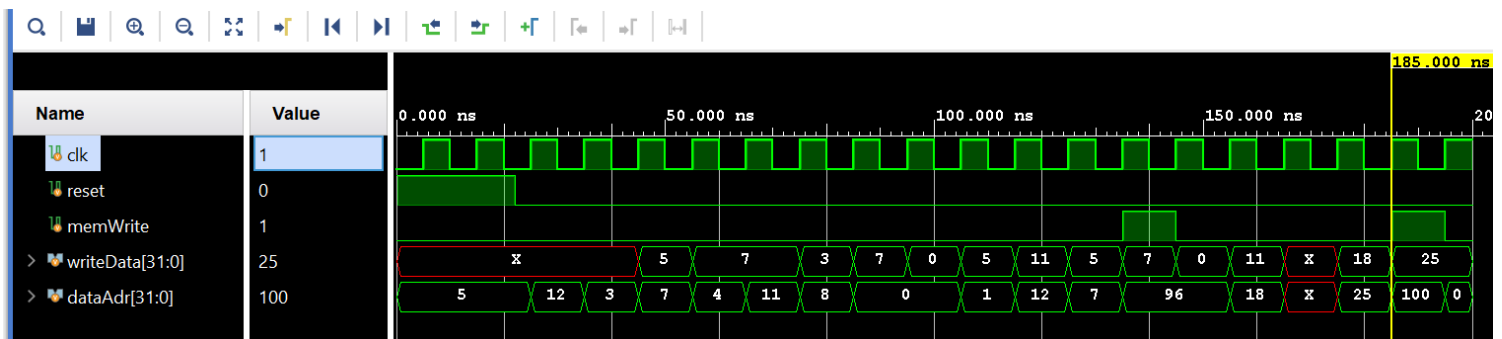


Figure 5.1: Vivado simulation waveform showing the successful execution of the test program.

Chapter 6: Pipelined Architecture

6.1. The 5-Stage Pipeline

To increase the instruction throughput, the single-cycle datapath was partitioned into a 5-stage pipelined architecture, as initially planned. The stages are:

1. Fetch (IF): The instruction is read from the Instruction Memory.
2. Decode (ID): The Register File is read, and the Control Unit decodes the instruction.
3. Execute (EX): The ALU performs arithmetic/logic operations or calculates branch targets.
4. Memory (MEM): Data Memory is accessed for Load/Store operations.
5. Writeback (WB): The final result is written back into the Register File.

6.2. Pipeline Registers

To isolate the data belonging to different instructions executing simultaneously, state registers were introduced between each stage: IF/ID, ID/EX, EX/MEM, and MEM/WB. These registers feature synchronous active-high reset and clear (clr) signals to allow dynamic flushing of the pipeline.

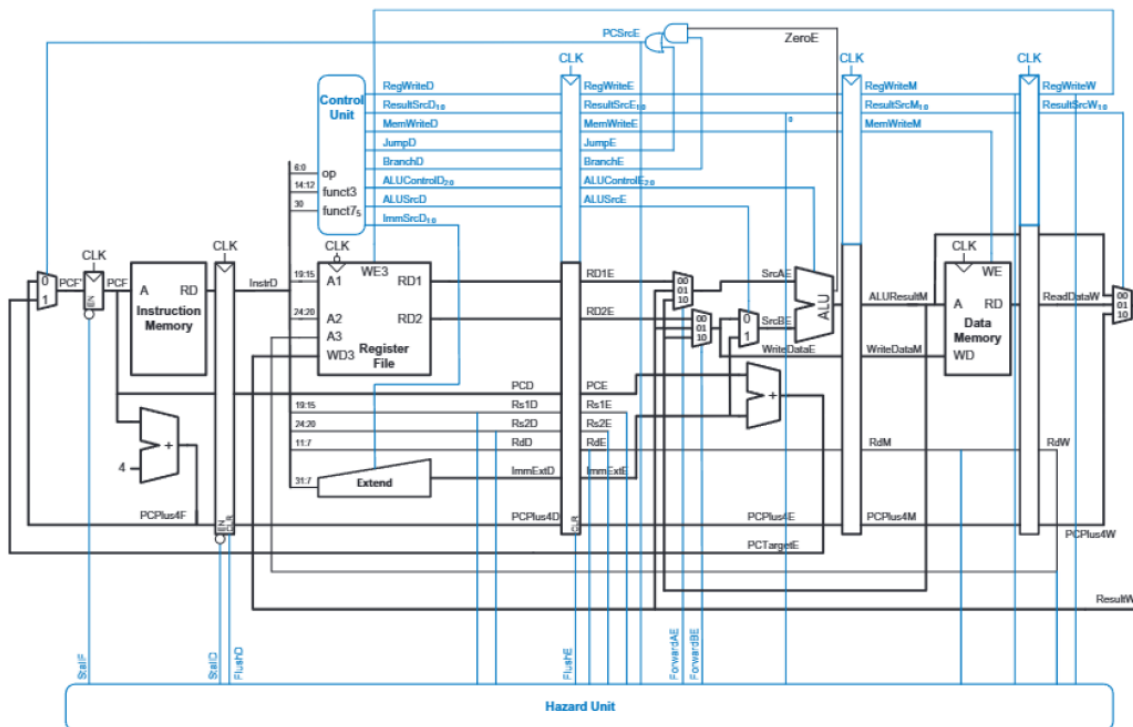


Figure 6.1: Pipelined processor with full hazard handling.

(Reference: Harris & Harris, *Digital Design and Computer Architecture RISC-V Edition*).

Chapter 7: Hazard Resolution (Hazard Unit)

Due to the overlapping execution of instructions, data and control dependencies arise. A dedicated Hazard Unit was implemented to ensure correct execution.

7.1. Data Hazards and Forwarding

When an instruction requires the result of a previous instruction that has not yet reached the Writeback stage, a data hazard occurs. The Hazard Unit detects if the source registers (rs1E, rs2E) in the Execute stage match the destination registers (rdM, rdW) of the instructions in the Memory or Writeback stages. If a match is found (and RegWrite is asserted), multiplexers forward the latest computed data directly to the ALU inputs, bypassing the Register File.

7.2. Load-Use Data Hazards and Stalling

If a Load Word (lw) instruction is followed immediately by an instruction that reads its result, forwarding alone is insufficient because the data is not available until the Memory stage. The Hazard Unit detects this by checking if the destination register of the lw in the Execute stage matches either source register of the instruction in the Decode stage. When detected, the pipeline is stalled for one cycle: the PC and IF/ID registers are frozen, and a no operation instruction (NOP) is inserted into the ID/EX register.

7.3. Control Hazards and Branch Flushing

When a branch instruction (e.g., beq) is taken, the instructions already fetched into the pipeline (following the branch) are invalid and must be discarded. The Hazard Unit asserts a flush signal to clear the IF/ID and ID/EX registers.

7.4. Hardware Design Considerations: NOP Injection

During a pipeline flush, simply zeroing out the IF/ID register (`instrD <= 0`) creates an invalid opcode (0000000). In Vivado, this causes the Main Decoder to output unknown states (X), propagating a fatal error throughout the datapath. To solve this, a true NOP instruction—`addi x0, x0, 0` (Machine Code: 00000013) is injected during a flush. This safely travels through the pipeline without altering the architectural state.

7.5. Pipeline Execution & Hazard Resolution Waveform

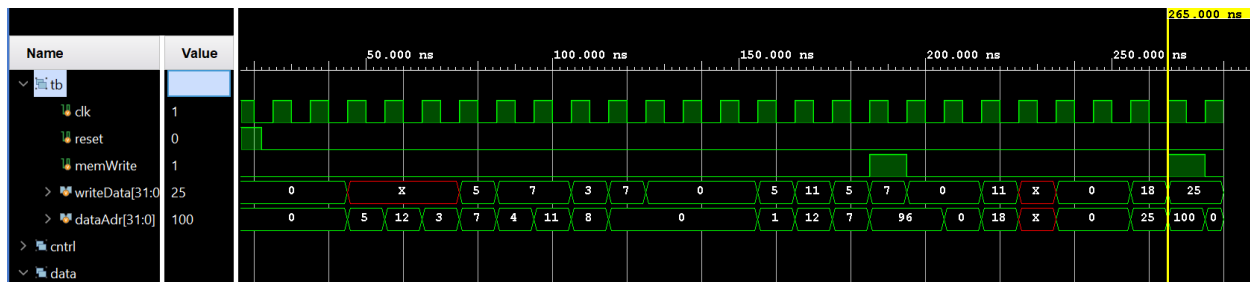


Figure 7.1: Simulation waveforms that present the pipelined processor working as expected.

The waveform above illustrates the successful validation of the 5-stage pipelined architecture using the top-level self-checking testbench. While the final memory transaction, writing the value 0x00000019 (25) to memory address 0x00000064 (100) is functionally identical to the single-cycle execution, the pipelined core requires approximately 265 ns to complete the benchmark program. This is a notable increase from the 185 ns completion time observed in the single-cycle variant. This extended execution timeline serves as a direct visual confirmation of the pipeline's temporal dynamics: it explicitly demonstrates the initial 5-cycle pipeline fill latency, coupled with the additional clock cycles consumed by the Hazard Unit when dynamically injecting stalls and flushes to safely resolve data and control dependencies.

This specific 'X', immediately following the data values of 11 and 18, visually demonstrates a successful Control Hazard resolution. Following the execution of the jump instruction (jal), the Hazard Unit triggers a flush to discard the erroneously fetched sequential instruction (addi x2, x0, 1). As this flushed state (pipeline bubble) propagates through the Execute stage, it manifests as an uninitialized data value. However, the architectural state remains completely safe.

Chapter 8: Conclusion

The development and implementation of the 32-bit RISC-V processor core provided a comprehensive, hands-on exploration of modern computer architecture principles. Transitioning the design from a theoretical schematic to a fully functional SystemVerilog implementation required deep integration of both hardware description logic and verification methodologies.

Through the iterative design process, several critical engineering milestones were achieved:

- **Functional Correctness:** Both the Single-Cycle and Pipelined architectures were successfully validated using a rigorous, self-checking testbench, correctly executing the RISC-V test assembly program and verifying memory write transactions.
- **Hazard Mitigation:** The integration of a dedicated Hazard Unit successfully resolved data dependencies through operand forwarding and handled control hazards via dynamic stalling and branch flushing.
- **Practical Hardware Insights:** Post-synthesis timing and performance analysis revealed the real-world trade-offs of microarchitectural design. While pipelining is theoretically superior, the physical overhead introduced by complex control routing and multiplexing on educational FPGA implementations highlighted the delicate balance between structural complexity and critical path optimization.

Beyond the specific scope of processor design, this project served as an invaluable learning experience that significantly strengthened core engineering competencies, such as systematic debugging and a deep understanding of low-level system execution. These transferable skills will provide a strong foundation for future challenges in my chosen career path in digital hardware engineering.